

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D4 — AI Usage Journal

Part 1 Submission

8 May 2026

Halit Batuhan ASLAN

220104004003

Journal Contents

Entry #1 — Domain Selection (Design) 29 April 2026

Entry #2 — Language Naming (Design) 1 May 2026

Entry #3 — Lexer Implementation (Lexer) 4 May 2026

Entry #4 — Parser Implementation (Parser) 4 May 2026

Entry #5 — Test Suite & D3 Programs (Testing) 4 May 2026

Experiment (E1) — EBNF Grammar Generation (Design/Parser) 1 May 2026

Entry # 1 of 5 · Date 2026-04-29 · Phase Design · AI tool Claude Opus 4.6

Chat Links: <https://claude.ai/share/109c8e7f-4e81-438c-9713-a1070507b8ef>

Goal. Select an application domain for my DSL that fits all project requirements and aligns with my interest in machine learning.

Prompt. "I want to design a language focusing on ML technologies. The goal is to perform configuration and training simulations of machine learning models — users can define models (KNN, Decision Tree, etc.), feed them into a training block, obtain results, compare metrics (analyzing overfitting/underfitting), and select the best model. The language will not run heavy ML libraries; it will simulate training and metric generation. Explore this domain and check if it matches all project requirements."

Response (key part). AI confirmed every project requirement maps naturally: 3+ primitive types (int, float, bool, metric), structured type (experiment record), control structures (if, experiment block), user-defined functions, and a genuine domain-specific construct (compare...select best by). It highlighted that the "simulation" angle gives strong Sebasta §3.5 operational semantics material, and the metric type creates a well-motivated name equivalence discussion for §6.14.

Accepted. The ML model selection & training simulation domain — it covers all minimum requirements without forcing any feature, and the "compare by ... select best" construct is a clear DSL-identity argument that no generic language would have.

Rejected / modified. Earlier broader ideas (a general ML scripting language) were eliminated because the rubric penalizes languages indistinguishable from generic scripting. I also rejected narrower options (pipeline-only, data-only) because they would make the expression language too thin for Sebasta Ch. 7 discussions.

Errors caught. None in this session — the AI's domain analysis was accurate. I verified the requirement mapping myself against the handout checklist.

Reflection. AI is very useful for brainstorming and validating domain scope against a rubric, but the final domain choice must come from my own interest and judgment. The AI helped me see why "too broad = generic language" and "too narrow = thin expressions" are both traps.

Entry # 2 of 5 · Date 2026-05-01 · Phase Design · AI tool Claude Opus 4.6

Chat Links: <https://claude.ai/share/109c8e7f-4e81-438c-9713-a1070507b8ef>

Goal. Choose a name for my DSL that reflects the ML model selection domain and incorporates an old Turkish word.

Prompt. "Give me some name ideas which correspond to ML model selection correctly and also an old Turkish word."

Response (key part). AI proposed several candidates: Ayar (Ottoman Turkish for "calibration/tuning"), Seçer ("selector"), Örnek ("sample/model"), Elgen ("intelligent"), Sezip ("to sense/detect"), Sınav ("trial/test"). Each was evaluated for domain fit, pronounceability, and file-extension viability.

Accepted. "Ayar" — it means calibration/tuning in Ottoman Turkish, which maps directly to hyperparameter tuning. Short, clean, works globally as a file extension (.ayar), and has a one-sentence defensible explanation.

Rejected / modified. Eliminated Seçer, Sınav, Örnek because they contain Turkish-specific characters (ç, ı, ö) making them inaccessible internationally. Eliminated Sezip (good sound but weak domain fit) and Elgen ("gen" part doesn't match the language's purpose precisely).

Errors caught. No factual errors. However, the AI did not proactively flag the Turkish character problem — I caught it myself and applied the elimination. This is a usability concern the AI should have anticipated given I said "address to world."

Reflection. AI generates creative options quickly but doesn't always apply practical constraints unless told. I should state all constraints (no special characters, international audience) upfront in naming prompts rather than filtering afterward.

Chat Links: <https://claude.ai/share/fa555c75-262b-4816-8429-1c44e20fa024>

Goal. Generate a complete lexical analyzer (Lexer) for Ayar in Python, consistent with the finalized EBNF grammar and D1 design specification.

Prompt. "I am a university student working on a CSE341 course project. My language is called Ayar — a DSL for ML model configuration, training simulation, and model selection. [Full EBNF grammar pasted]. Write a clean, object-oriented Lexer class in Python. No external libraries (PLY, Lark, ANTLR). Strict requirements: Token dataclass with type/value/line/column; explicit keyword vs identifier separation; INT vs FLOAT distinction; comment and whitespace handling with correct line/column tracking; custom LexicalError with exact location; all separators and operators including two-char ones (`==`, `!=`, `<=`, `>=`). Include a test block with my sample program."

Response (key part). AI produced a well-structured ~180-line lexer with six sections: Token dataclass, LexicalError exception, keyword/metric/bool frozensets, a compiled master regex (TOKEN_SPEC), the tokenize() loop, and a sample program test. It used a single compiled regex with named groups and maximal-munch ordering. The WORD pattern matches all identifier-shaped strings, then a post-classification step checks frozensets to assign KEYWORD, METRIC_LITERAL, BOOL_LITERAL, or IDENTIFIER types. It produced 236 tokens from the sample program with no errors.

Accepted. The overall architecture — single compiled master regex with ordered alternatives implementing maximal munch (Sebesta §4.2), post-scan keyword classification via frozensets (Sebesta §3.3), Token dataclass with line/column tracking, and custom LexicalError. The pattern ordering logic was correct: COMMENT before OP_DIV, FLOAT before INT, two-char operators before single-char ones. The 10-test verification suite was also useful for confirming each token category.

Rejected / modified. (1) Identifier regex was `[A-Za-z_][A-Za-z0-9_]*` — allowing underscore as a first character. My EBNF says `<identifier> ::= <letter> { <letter> | <digit> | "_" }`, so underscore can only appear inside, not at the start. Fixed to `[A-Za-z][A-Za-z0-9_]*`. (2) The sample program included a NaiveBayes model with `var_smoothing = 1e-9` — scientific notation does not exist in my grammar's float_literal rule. Removed that model entirely. (3) The analyze block in the sample used `threshold = 0.05;` (semicolon followed by comma) but my EBNF's analyze_field_list has no comma separator — fields are simply juxtaposed, each self-terminated with `;`. Fixed the sample accordingly.

Errors caught. (1) Underscore-start bug — `_myvar` would have been silently accepted as a valid identifier, violating my lexical spec. Caught by tracing the EBNF identifier rule against the regex. (2) Scientific notation `1e-9` is not in the grammar but was used in the test program; the lexer would split it into four tokens (INT "1", IDENTIFIER "e", OP_MINUS "-", INT "9") which would then confuse the parser. Caught by cross-referencing the float_literal EBNF rule. (3) Comma-in-analyze was a sample-level error, not a lexer bug — the lexer would tokenize it fine, but the parser would reject it. Caught by comparing the sample against the finalized EBNF.

Reflection. AI produces solid lexer architecture quickly — the master-regex approach with maximal munch and post-classification is textbook-correct. However, it does not rigorously cross-check the regex patterns against the EBNF rules character by character; the underscore-start and scientific notation issues both came from the AI applying "common Python

conventions" rather than reading my specific grammar. Lesson: always diff every regex in TOKEN_SPEC against the corresponding EBNF lexical rule before accepting.

Chat Links: <https://claude.ai/share/77689341-3415-449d-8bbc-44307aa38130>

Goal. Generate a recursive-descent parser for Ayar that builds an AST, consistent with the finalized EBNF grammar and my existing lexer.

Prompt. "I am designing a DSL called Ayar for ML model configuration, training simulation, and model selection. I previously wrote the Lexer. Now write a clean, object-oriented Recursive-Descent Parser in Python. No external libraries. [Full EBNF grammar pasted]. Strict requirements: explicit AST node classes (ProgramNode, ModelDeclNode, BinOpNode, etc.); one parsing method per EBNF non-terminal; left-associative AST construction for arithmetic/logical operators; custom ParseError with expected token, actual token, and exact line/column; AST dump method for readable indented output. Assume tokens have .type, .value, .line_number, .column_number attributes."

Response (key part). AI produced a ~500-line parser with 20+ AST node classes, one `_parse_*` method per non-terminal, iterative while-loops for left-associativity (Sebesta §4.4), LL(1) dispatch via keyword lookahead, and a `_consume()` gate method that raises `ParseError` with full location info. It included a self-test block and passed on the sample program.

Accepted. The core recursive-descent architecture — one method per non-terminal with LL(1) dispatch — is textbook-correct (Sebesta §4.4.1). The left-associativity mechanism using while-loops (accumulating the left subtree as each operator is consumed) correctly avoids the infinite recursion that direct left-recursion would cause in a top-down parser. The `ParseError` design with exact location metadata via `_consume()` was also clean. Additionally, I accepted the layout validation pass (`validate_layout`) that enforces my curly-brace-on-its-own-line rule as a pre-parse step — this was a good separation of concerns since line-position constraints cannot be expressed in EBNF.

Rejected / modified. (1) Token type names mismatched my lexer in 8 places: AI used `ASSIGN` (my lexer emits `OP_ASSIGN`), `INT_LIT/FLOAT_LIT/STRING_LIT` (my lexer emits `INT_LITERAL/FLOAT_LITERAL/STRING_LITERAL`), and treated booleans/metrics as keywords rather than dedicated `BOOL_LITERAL` and `METRIC_LITERAL` types. All 8 were corrected by cross-referencing the parser's `_consume()` calls against my lexer's actual output. (2) `_parse_split_part` accepted `INT_LITERAL` for ratios — my EBNF says `<float_literal>` only, so `train(1)` should be a `ParseError`. Fixed to reject integers. (3) `_parse_analyze_stmt` used commas between fields — my EBNF's `analyze_field_list` has no comma separator, fields are juxtaposed and each self-terminated with `;`. Rewrote the loop to continue while the next token is a recognized field keyword. (4) `_parse_eval_target` only handled plain identifier and bracket form — missing the dotted form comparison.best shown in D1's sample program. Added it as a third alternative.

Errors caught. (1) The 8 token-type mismatches caused immediate test failures — every `_consume("ASSIGN")` call crashed because the lexer emits `OP_ASSIGN`. Caught by running the parser with my actual lexer. (2) The split ratio bug would silently accept `train(1)` as valid, violating the grammar. Caught by tracing the EBNF rule. (3) The comma-in-analyze bug would reject my D1 sample program's `analyze` block. Caught by parsing the sample program end-to-end.

Reflection. AI produces excellent parser architecture (recursive-descent structure, precedence-via-call-depth, left-associativity via iteration) but is unreliable on integration details — it assumed token type names rather than reading my actual lexer output. The most important

lesson: always run the AI-generated parser against your own lexer's real token stream before accepting anything, and cross-check every `_consume()` call against the lexer's token type catalog.

Chat Links: <https://claude.ai/share/c3142c7b-733c-4111-8055-8aa7458a9670>

Goal. Generate valid and invalid Ayar test programs (D3 deliverable) and a test runner script to verify the lexer and parser work correctly end-to-end.

Prompt. "I have attached my Lexer, Parser, and D1 Design Spec with EBNF grammar. My next deliverable is D3 — Example Programs & Test Report. Generate 3 valid .ayar programs that together exercise every language feature (dataset, splits, all model types, all experiment statements, report). Generate 5 invalid .ayar programs each with a deliberate syntax error. Write a run_tests.py script that imports my Lexer and Parser, tests valid programs by printing the AST, and tests invalid programs by catching LexicalError/ParseError with line and column info."

Response (key part). AI produced 3 valid programs (sample_1 covering KNN+DecisionTree with bracket eval, sample_2 covering SVM+NaiveBayes+LogisticRegression with lower_is_better, sample_3 covering all five model types), 5 invalid programs (missing semicolon, unknown model type, invalid split role, inline brace layout violation, unknown keyword), and a 34-test runner script. All passed initially.

Accepted. The test runner architecture — a simple custom script with no external frameworks, using try/except to catch ParseError and LayoutError with fragment-matching assertions. The 3 valid programs together cover every grammar rule: all 5 model types, all split roles, all experiment statements (train, evaluate, collect, analyze, compare, exclude, select), both compare directions, all three eval_target forms (plain, dotted, bracket), and both report field types. The isolation tests (V1–V12) testing one feature each were also useful for pinpointing failures.

Rejected / modified. (1) The initial test files used commas between analyze fields (threshold = 0.10;,) because the AI based them on an older parser version. After I updated the parser to drop commas from analyze_field_list (fields now juxtaposed, each self-terminated with ;), all analyze blocks in the 3 valid programs and all inline tests had to be regenerated. (2) The initial tests did not cover the dotted eval_target form (evaluate comparison.best on iris.test;) because the first parser version lacked it. After I added that form to the parser, a new valid test (V9) was added and sample_1 was updated to use it. (3) I had to re-request the full test suite twice because the AI kept generating tests against a stale version of the parser rather than re-reading the uploaded files from scratch.

Errors caught. (1) The AI's first test suite passed 33/33 against an old parser but failed against my updated one — specifically every analyze block test broke because the old ;, separator was now a ParseError. Caught by running the tests after updating the parser. (2) An invalid test case (I11 — empty analyze body) was missing initially; the updated parser explicitly raises an error for analyze overfitting { } with no fields, so a dedicated failure test was needed. Caught by manually testing edge cases.

Reflection. AI generates comprehensive test suites quickly, but they become stale the moment you change the parser or grammar. The key lesson: after any EBNF or parser modification, re-run the entire test suite immediately and re-generate affected tests. Also, providing the AI with both the grammar and the actual parser source (not just the grammar) is essential — otherwise it generates tests against what the grammar says, not what the code actually does.

Experiment 1 -> E1 · Date 2026-05-01 · Phase Parser / Design · AI tool Claude Opus 4.6

Chat Links: <https://claude.ai/share/9ef8e86e-e704-49ac-8260-a7a55046f212>

Goal: Generate complete EBNF grammar for Ayar following Sebesta §3.1–3.2 style (Experiment E1).

Prompt. "I am a university student working on a CSE341 course project. My language is called Ayar — a DSL for ML model configuration, training simulation, and model selection. [Full example program pasted]. Design decisions already fixed: primitive types int, float, bool, metric; structured type experiment; static scoping; name equivalence for metric; int→float coercion only; assignment is a statement; short-circuit for or/and; intended precedence low→high: or → and → not → == != < <= > >= → + - → * / → unary minus → primary. YOUR TASK — Generate a complete unambiguous EBNF grammar covering: program structure, dataset/split, model declaration, experiment block (train, evaluate, collect, analyze, compare, exclude, select), report, expression language with correct precedence, all types/literals. After generating, provide: operator precedence table, ambiguities found, assumptions list, and two design decisions to reconsider."

Response (key part). AI produced a full EBNF grammar with 40+ productions using stratified nonterminals for expression precedence (eight levels deep). It identified 5 ambiguities (most notably the or keyword collision between exclude_condition and boolean expressions) and listed 10 assumptions (A1–A10). It recommended two design changes: (D1) make model types open identifiers instead of grammar-level keywords, (D2) drop the bracket syntax [comparison.best] in favor of plain dot notation.

Accepted. The overall stratified expression grammar structure encoding precedence through nonterminal depth — this correctly implements Sebesta §3.3's technique. The or keyword collision resolution (giving exclude_condition its own closed sub-grammar separate from <expr>) was a genuine ambiguity I had missed. The recommendation D2 to simplify eval_target notation was also adopted.

Rejected / modified. (1) The AI used commas as separators inside the analyze block, but my parser actually uses semicolons as terminators on each field with no separator between fields. I corrected analyze_field_list to ::= <analyze_field> { <analyze_field> } and analyze_field to end with ";". (2) The AI's initial grammar omitted the plain dotted form <identifier> "." <identifier> from <eval_target> — only had plain identifier and bracketed form. I added the dotted form as a middle alternative so the parser tries the longer match first. (3) I kept model types as closed keywords (rejected D1) because my parser validates them at syntax level and I can defend this choice as a deliberate DSL constraint in the exam.

Errors caught. (1) Separator inconsistency in analyze block — AI applied the comma convention from model blocks without checking that analyze uses a different pattern (semicolon-terminated, no separator). Caught by comparing the generated EBNF against my actual parser code. (2) Missing eval_target alternative — the dotted form comparison.best was absent, meaning a valid program would be rejected. Caught by tracing a test program through the grammar by hand.

Reflection. AI generates structurally sound grammars quickly, especially the precedence-encoding layered nonterminals, but it unifies separator conventions across all blocks even when the actual design differs block by block. The lesson: always trace at least one complete example

program through the generated grammar rule-by-rule before accepting it. Also, providing my own precedence table in the prompt made error detection much easier — without it I would not have had a clear baseline to compare against.